

Міністерство освіти і науки України

Національний технічний університет України

Київський політехнічний інститут імені Ігоря Сікорського

Лабораторно робота № 3

«Розвертання Linux Kernel в середовищі програмного емулятора QEMU для архітектури процесорів ARM»

Роботу виконав

Студент групи ІО-46

Павленко Артур Андрійович

Роботу перевірив:

Каплунов Артем Володимирович

Київ 2026

Посилання на репозиторій:

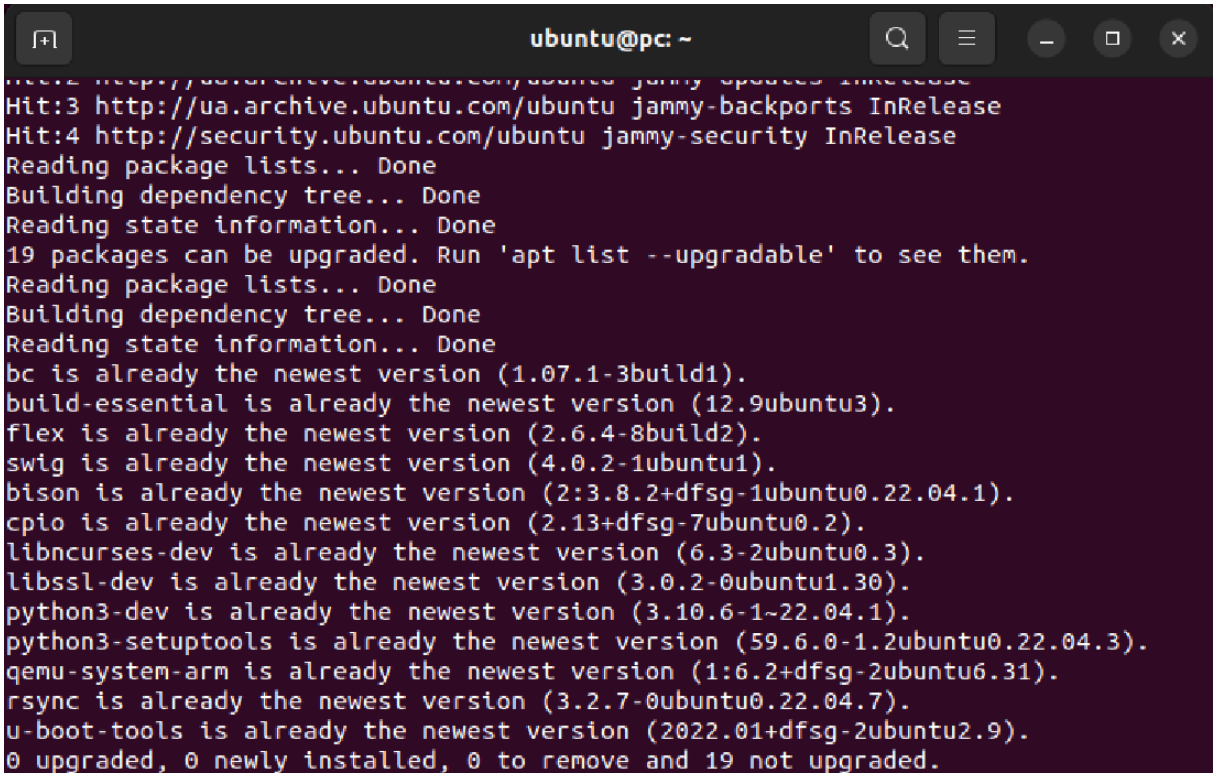
https://github.com/Reaftepi/AK2/tree/main/Lab3

Хід роботи

1. Підготовка програмного середовища

Перед збиранням я встановив пакети, потрібні для компіляції U-Boot, ядра Linux і BusyBox, а також емулятор QEMU для архітектури ARM.

```
sudo apt update
sudo apt install build-essential bison flex bc libssl-dev libncurses-dev \
    u-boot-tools cpio rsync swig python3-dev python3-setuptools \
    qemu-system-arm
```



Скріншот 1 (пакети вже були встановлені)

Джерельні коди U-Boot та ядра версій, що використовуються в інструкції, містять утиліту dtc. Компілятор хоста gcc 10 і новіших версій під час її збирання видає помилку «multiple definition of yylloc». Тому я призначив хостовим компілятором gcc 9 через механізм update-alternatives:

```
gcc --version
sudo apt install gcc-9
```

sudo update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-11 11
sudo update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-9 9
sudo update-alternatives --config gcc
gcc --version

```
There are 2 choices for the alternative gcc (providing /usr/bin/gcc).

  Selection    Path             Priority   Status
------------------------------------------------------------
* 0            /usr/bin/gcc-11   11        auto mode
  1            /usr/bin/gcc-11   11        manual mode
  2            /usr/bin/gcc-9    9         manual mode

Press <enter> to keep the current choice[*], or type selection number: 2
update-alternatives: using /usr/bin/gcc-9 to provide /usr/bin/gcc (gcc) in manua
l mode
gcc (Ubuntu 9.5.0-1ubuntu1~22.04.1) 9.5.0
Copyright (C) 2019 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Скріншот 2

Важлива деталь: зміна стосується лише компілятора хоста, яким збираються допоміжні утиліти. Код для ARM компілюється крос-тулчейнами з каталогу /opt.

Я перевірив наявність і працездатність обох крос-компіляторів:

ls /opt
/opt/gcc-arm-8.3-2019.03-x86_64-arm-eabi/bin/arm-eabi-gcc --version
/opt/gcc-arm-8.3-2019.03-x86_64-arm-linux-gnueabihf/bin/arm-linux-gnueabihf-gcc --version

```
gcc-arm-8.3-2019.03-x86_64-arm-eabi             VBoxGuestAdditions-7.2.2
gcc-arm-8.3-2019.03-x86_64-arm-linux-gnueabihf
arm-eabi-gcc (GNU Toolchain for the A-profile Architecture 8.3-2019.03 (arm-rel-
8.36)) 8.3.0
Copyright (C) 2018 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

arm-linux-gnueabihf-gcc (GNU Toolchain for the A-profile Architecture 8.3-2019.0
3 (arm-rel-8.36)) 8.3.0
Copyright (C) 2018 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Скріншот 3

Я створив робочий каталог для джерельних кодів і перейшов до нього:

mkdir -p ~/repos && cd ~/repos

```
ubuntu@pc:~$ mkdir -p ~/repos && cd ~/repos
ubuntu@pc:~/repos$
```

Скріншот 4

2. Збирання завантажувача U-Boot (пункт 2.1)

Я завантажив джерельний код U-Boot і перейшов на стабільну версію:

cd ~/repos
git clone https://source.denx.de/u-boot/u-boot.git
cd u-boot
git checkout v2019.07

```
ubuntu@pc:~/repos$ git clone https://source.denx.de/u-boot/u-boot.git
Cloning into 'u-boot'...
warning: redirecting to https://git.u-boot-project.org//u-boot/u-boot.git/
remote: Enumerating objects: 1137641, done.
remote: Counting objects: 100% (1887/1887), done.
remote: Compressing objects: 100% (511/511), done.
remote: Total 1137641 (delta 1374), reused 1878 (delta 1369), pack-reused 113575
4 (from 1)
Receiving objects: 100% (1137641/1137641), 228.67 MiB | 561.00 KiB/s, done.
Resolving deltas: 100% (942037/942037), done.
Updating files: 100% (38856/38856), done.
ubuntu@pc:~/repos$ cd u-boot
ubuntu@pc:~/repos/u-boot$ git checkout v2019.07
Updating files: 100% (41168/41168), done.
Note: switching to 'v2019.07'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at e5aee22e4be Prepare v2019.07
ubuntu@pc:~/repos/u-boot$
```

Скріншот 5

Для збирання завантажувача я використав bare-metal тулчейн arm-eabi. Щоб прискорити повторні збірки, я налаштував виклик компілятора через ccache:

export PATH=/opt/gcc-arm-8.3-2019.03-x86_64-arm-eabi/bin:$PATH
export CROSS_COMPILE='ccache arm-eabi-'
export ARCH=arm

Я застосував конфігурацію для SoC AM335x (BeagleBone Black) і запустив збирання:

make am335x_evm_defconfig
make -j4

```
ubuntu@pc:~/repos/u-boot$ export PATH=/opt/gcc-arm-8.3-2019.03-x86_64-arm-eabi/b
in:$PATH
ubuntu@pc:~/repos/u-boot$ export CROSS_COMPILE='ccache arm-eabi-'
ubuntu@pc:~/repos/u-boot$ export ARCH=arm
ubuntu@pc:~/repos/u-boot$ echo $CROSS_COMPILE
ccache arm-eabi-
ubuntu@pc:~/repos/u-boot$ arm-eabi-gcc --version
arm-eabi-gcc (GNU Toolchain for the A-profile Architecture 8.3-2019.03 (arm-rel-
8.36)) 8.3.0
Copyright (C) 2018 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

ubuntu@pc:~/repos/u-boot$ make am335x_evm_defconfig
  HOSTCC  scripts/basic/fixdep
  HOSTCC  scripts/kconfig/conf.o
  YACC    scripts/kconfig/zconf.tab.c
  LEX     scripts/kconfig/zconf.lex.c
  HOSTCC  scripts/kconfig/zconf.tab.o
  HOSTLD  scripts/kconfig/conf
#
# configuration written to .config
#
ubuntu@pc:~/repos/u-boot$ make -j4
scripts/kconfig/conf  --syncconfig Kconfig
  CHK     include/config.h
  UPD     include/config.h
```

Скріншот 6

```
  OBJCOPY spl/u-boot-spl-nodtb.bin
  COPY    spl/u-boot-spl.bin
  MKIMAGE MLO
  MKIMAGE MLO.byteswap
=================== WARNING ======================
This board does not use CONFIG_WDT (DM watchdog support).
Please update the board to use CONFIG_WDT before the
v2019.10 release.
Failure to update by the deadline may result in board removal.
See doc/driver-model/MIGRATION.txt for more info.
==================================================
  CFGCHK  u-boot.cfg
ubuntu@pc:~/repos/u-boot$
```

Скріншот 7

Примітка: наприкінці збирання з'явилося попередження про міграцію коду плати на нову модель драйверів (CONFIG_WDT). Воно адресоване розробникам U-Boot і не є помилкою збирання.

Я перевірив наявність результатів збирання: MLO (вторинний завантажувач SPL) та u-boot.img:

ls -l MLO u-boot.img

```
ubuntu@pc:~/repos/u-boot$ ls -l MLO u-boot.img
-rw-rw-r-- 1 ubuntu ubuntu 110180 жов 19 17:49 MLO
-rw-rw-r-- 1 ubuntu ubuntu 750896 жов 19 17:49 u-boot.img
ubuntu@pc:~/repos/u-boot$
```

Скріншот 8

3. Збирання ядра Linux (пункт 2.2)

Я завантажив джерельний код стабільної гілки ядра. Щоб не завантажувати всю історію репозиторію, я використав неглибоке клонування:

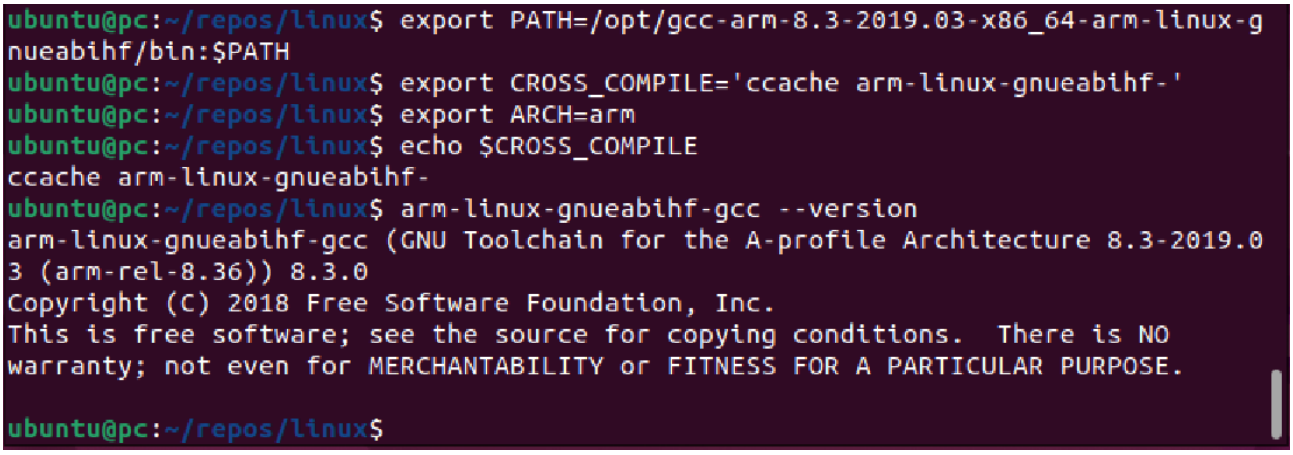
cd ~/repos
*git clone --depth 1 -b linux-5.4.y *
https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git linux
cd linux

```
ubuntu@pc:~/repos$ git clone --depth 1 -b linux-5.4.y \
    https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git linux
Cloning into 'linux'...
remote: Enumerating objects: 69794, done.
remote: Counting objects: 100% (69794/69794), done.
remote: Compressing objects: 100% (68111/68111), done.
remote: Total 69794 (delta 4467), reused 15048 (delta 872), pack-reused 0 (from
0)
Receiving objects: 100% (69794/69794), 187.42 MiB | 5.95 MiB/s, done.
Resolving deltas: 100% (4467/4467), done.
Updating files: 100% (65746/65746), done.
ubuntu@pc:~/repos$ cd linux
ubuntu@pc:~/repos/linux$
```

Скріншот 9

Ядро і програми користувацького простору збираються тулчейном arm-linux-gnueabihf, тому я перевизначив змінні середовища:

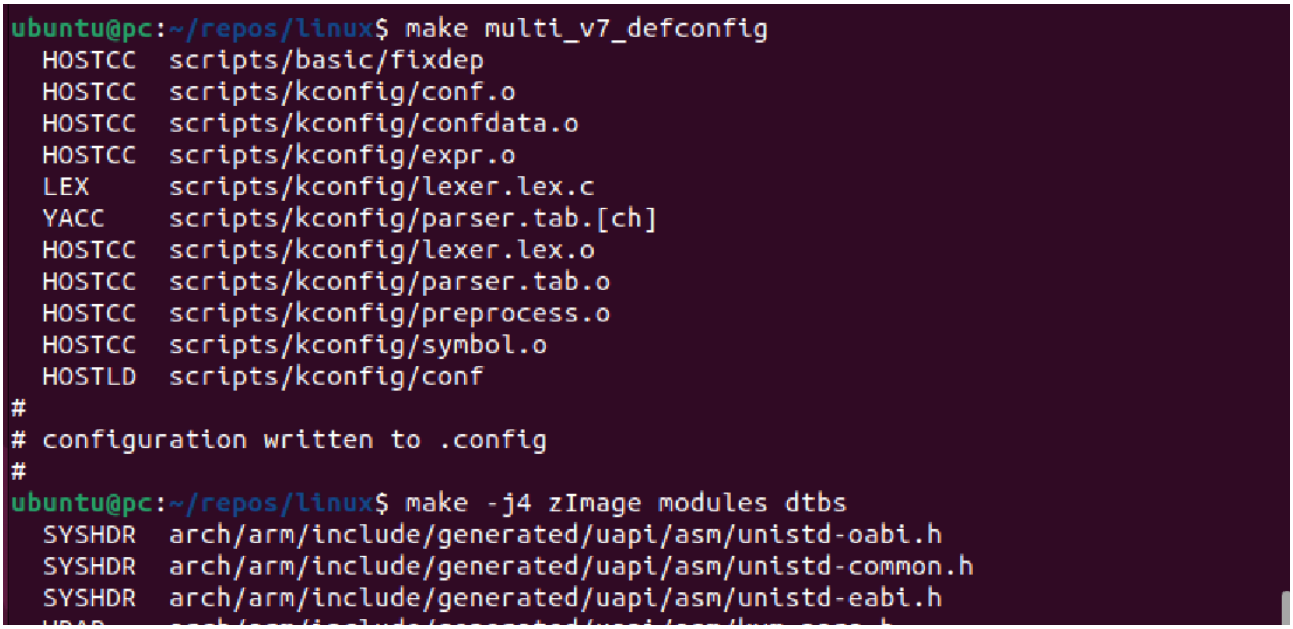
export PATH=/opt/gcc-arm-8.3-2019.03-x86_64-arm-linux-gnueabihf/bin:$PATH
export CROSS_COMPILE='ccache arm-linux-gnueabihf-'
export ARCH=arm



Скріншот 10

Я застосував конфігурацію multi_v7_defconfig і зібрав образ ядра, модулі та дерева пристроїв:

make multi_v7_defconfig
make -j4 zImage modules dtbs

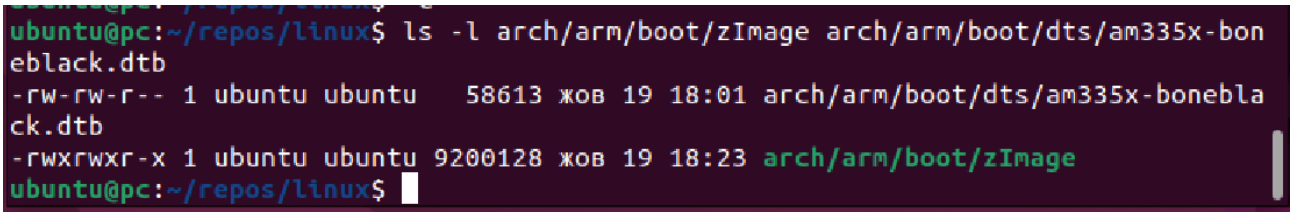


Скріншот 11

Важлива деталь: QEMU не емулює SoC AM335x, тому система запускається на віртуальній платі virt. Конфігурація multi_v7_defconfig підтримує і плату virt, і AM335x. Отже, це саме ядро придатне для завантаження на реальній BeagleBone Black.

Я перевірив наявність образу ядра та дерева пристроїв для BeagleBone Black:

ls -l arch/arm/boot/zImage arch/arm/boot/dts/am335x-boneblack.dtb

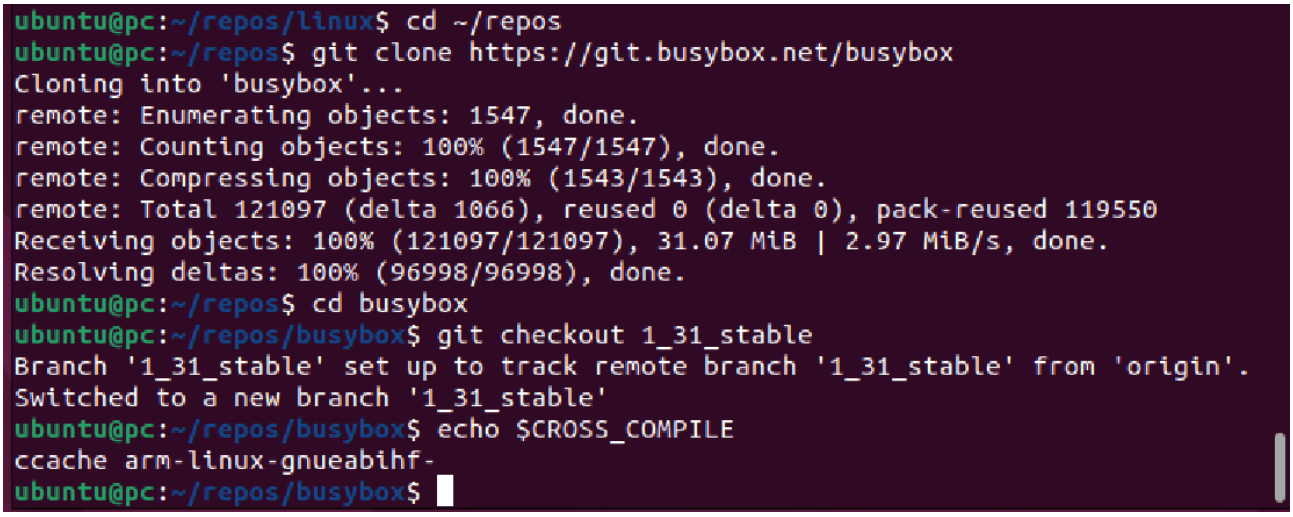


Скріншот 12

4. Створення кореневої файлової системи на основі BusyBox (пункт 2.3)

Я завантажив джерельний код BusyBox і перейшов на стабільну гілку:

cd ~/repos
git clone https://git.busybox.net/busybox
cd busybox
git checkout 1_31_stable



Скріншот 13

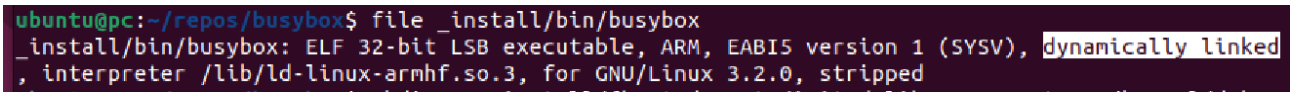
Змінні PATH, CROSS_COMPILE та ARCH залишилися такими самими, як для ядра (тулчейн arm-linux-gnueabihf). Я створив конфігурацію за замовчуванням і зібрав BusyBox з динамічним компонуванням:

make defconfig

make -j4

make install

file _install/bin/busybox



Скріншот 14

У каталозі _install я створив структуру кореневої файлової системи:

mkdir -p _install/{boot,dev,etc/init.d,lib,proc,root,sys/kernel/debug,tmp}

Я створив сценарій ініціалізації etc/init.d/rcS, який монтує sysfs, procfs і debugfs та запускає mdev для автоматичного створення файлів пристроїв. Потім я зробив його виконуваним і створив посилання /init на BusyBox. Вміст сценарію:

#!/bin/sh

mount -t sysfs none /sys

mount -t proc none /proc

mount -t debugfs none /sys/kernel/debug

echo /sbin/mdev > /proc/sys/kernel/hotplug

mdev -s

chmod +x _install/etc/init.d/rcS

ln -s bin/busybox _install/init

Я скопіював у /boot образ ядра, дерево пристроїв, таблицю символів і конфігурацію ядра:

cd _install/boot

cp ~/repos/linux/arch/arm/boot/zImage .

cp ~/repos/linux/arch/arm/boot/dts/am335x-boneblack.dtb .

cp ~/repos/linux/System.map .

cp ~/repos/linux/.config ./config

cd -

Я встановив модулі ядра в rootfs і скопіював бібліотеки тулчейна, потрібні динамічному BusyBox:

cd ~/repos/linux

export INSTALL_MOD_PATH=~/repos/busybox/_install

make modules_install

cd -

arm-linux-gnueabihf-readelf -d _install/bin/busybox | grep NEEDED

cd _install/lib

libc_dir=$(arm-linux-gnueabihf-gcc -print-sysroot)/lib

cp -a $libc_dir/.so* .*

cd -

Я створив у /etc конфігурацію mdev, облікові записи root і налаштування DNS:

echo '$MODALIAS=. root:root 660 @modprobe "$MODALIAS"' > _install/etc/mdev.conf*

echo 'root:x:0:' > _install/etc/group

echo 'root:x:0:0:root:/root:/bin/sh' > _install/etc/passwd

echo 'root::10933:0:99999:7:::' > _install/etc/shadow

echo "nameserver 8.8.8.8" > _install/etc/resolv.conf

5. Завантаження системи в емуляторі QEMU (пункт 3.1)

Я запакував кореневу файлову систему в архів initramfs формату cpio:

cd ~/repos/busybox/_install

find . | cpio -o -H newc | gzip > ../rootfs.cpio.gz

cd ..

Я запустив ядро та rootfs у QEMU на віртуальній ARM-платі virt з 512 МБ оперативної пам'яті. Плата virt не потребує файлу dtb, а завантажувач для QEMU не потрібен, тому образи U-Boot тут не використовуються. Rootfs монтується як RAM-диск:

*qemu-system-arm -kernel _install/boot/zImage -initrd rootfs.cpio.gz *

*-machine virt -nographic -m 512 *

--append "root=/dev/ram0 rw console=ttyAMA0,115200 mem=512M"

Ядро розпакувало rootfs і запустило процес /init. Після натискання Enter мені стала доступна командна оболонка BusyBox.

Для перевірки коректності я виконав команди з кінця пункту 3.1 інструкції:

uname -a

ls -l

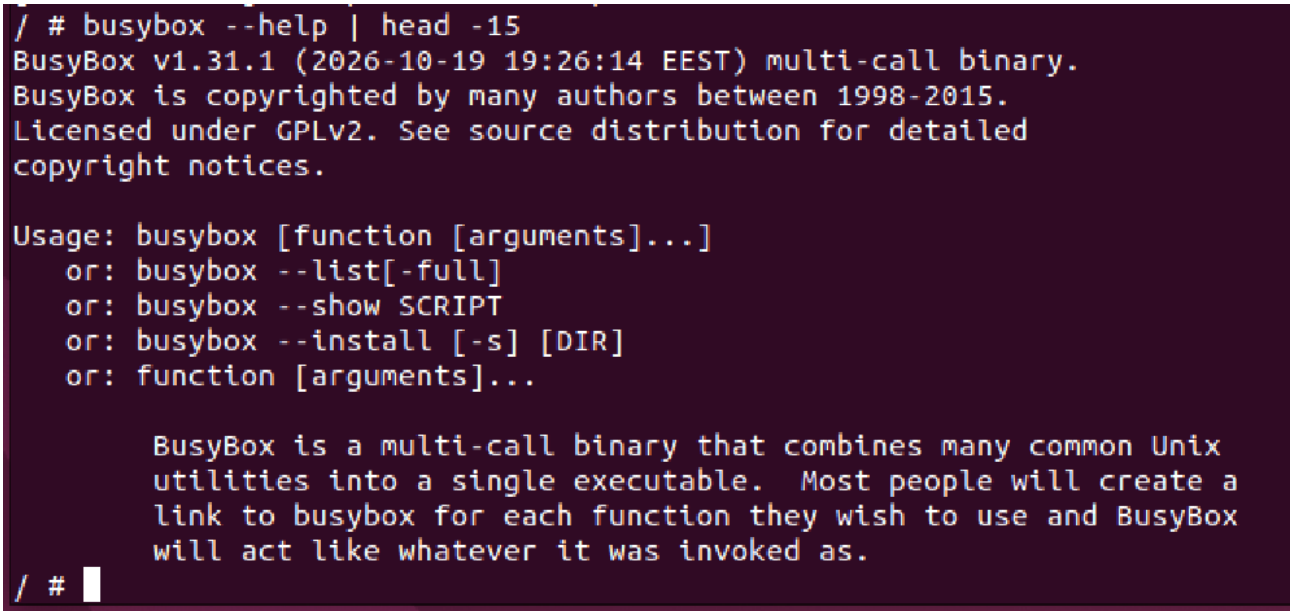
dmesg | grep init

busybox --help | head -15

poweroff

```
/ # uname -a
Linux (none) 5.4.302 #1 SMP Mon Oct 19 18:00:28 EEST 2026 armv7l GNU/Linux
/ # ls -l
total 0
drwxrwxr-x    2 1000     1000             0 Oct 19 16:27 bin
drwxrwxr-x    2 1000     1000             0 Oct 19 16:27 boot
drwxrwxr-x    3 1000     1000             0 Oct 19 16:42 dev
drwxrwxr-x    3 1000     1000             0 Oct 19 16:28 etc
lrwxrwxrwx    1 1000     1000            11 Oct 19 16:27 init -> bin/busybox
drwxrwxr-x    3 1000     1000             0 Oct 19 16:28 lib
lrwxrwxrwx    1 1000     1000            11 Oct 19 16:27 linuxrc -> bin/busybox
dr-xr-xr-x   88 root     root             0 Jan  1  1970 proc
drwxrwxr-x    2 1000     1000             0 Oct 19 16:27 root
drwxrwxr-x    2 1000     1000             0 Oct 19 16:27 sbin
dr-xr-xr-x   12 root     root             0 Oct 19 16:42 sys
drwxrwxr-x    2 1000     1000             0 Oct 19 16:27 tmp
drwxrwxr-x    4 1000     1000             0 Oct 19 16:27 usr
/ # dmesg | grep init
[    0.000000] mem auto-init: stack:off, heap alloc:off, heap free:off
[    0.000000] Memory: 403824K/524288K available (12288K kernel code, 1664K rwdata, 5252K rodata
, 2048K init, 395K bss, 54928K reserved, 65536K cma-reserved, 0K highmem)
[    0.041193] devtmpfs: initialized
[    0.059480] pinctrl core: initialized pinctrl subsystem
[    0.196782] SCSI subsystem initialized
[    0.558220] Trying to unpack rootfs image as initramfs...
[    1.678928] Freeing initrd memory: 26608K
[    1.788625] SuperH (H)SCI(F) driver initialized
[    1.790672] msm_serial: driver initialized
[    1.791239] STMicroelectronics ASC driver initialized
[    1.792606] STM32 USART driver initialized
[    2.993549] Run /init as init process
```

Скріншот 15



Скріншот 16

Після перевірки я вимкнув систему командою poweroff і зберіг історію виконаних команд у файл:

cd ~ && date && history > history3.txt

Висновок

У ході роботи я зібрав усі програмні компоненти для запуску ОС Linux на процесорах архітектури ARM. Це завантажувач U-Boot для SoC AM335x, ядро Linux з конфігурацією multi_v7_defconfig і коренева файлова система на основі динамічно зібраного BusyBox з бібліотеками тулчейна та модулями ядра. Збирання я виконував крос-тулчейнами gcc-arm-8.3 з кешуванням через ccache. Для сумісності утиліти dtc я призначив хостовим компілятором gcc 9 за допомогою update-alternatives.

Я сформував rootfs зі сценарієм ініціалізації rcS, утилітою mdev для автоматичного створення пристроїв і каталогами /boot, /lib та /etc, і запакував її в архів initramfs формату cpio. Систему я успішно завантажив в емуляторі QEMU на віртуальній платі virt. Команди перевірки підтвердили роботу зібраного ядра 5.4.302 на архітектурі ARMv7 (armv7l) і запуск BusyBox 1.31.1 як процесу /init. Отримані навички дозволяють мені надалі перенести ту саму збірку на реальну плату BeagleBone Black через SD-карту або eMMC.